

Week 5, Notebook 2: Transformers with PyTorch

Build, Train, and Use Transformer Models

What you'll build: A Transformer-based sequence classifier and explore pre-trained models with Hugging Face.

New concepts:

- `nn.TransformerEncoder` in PyTorch
- Causal masking for autoregressive models
- Transfer learning: using pre-trained Transformers
- Tokenization and the Hugging Face ecosystem

Time estimate: 60 minutes

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import math
8
9 torch.manual_seed(42)
10 np.random.seed(42)
11 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
12 print(f"Device: {device}")
```

Part 1: Transformer Encoder for Sequence Classification

In [2]

```

1 # =====
2 # Transformer encoder for classifying sequences
3 # Task: classify sequences of numbers as "ascending" or "not"
4 # =====
5
6 def generate_sequence_data(n_samples=2000, seq_len=8, vocab_size=50):
7     """Generate sequences and labels: 1=ascending, 0=random."""
8     x = []
9     y = []
10    for _ in range(n_samples):
11        if np.random.rand() > 0.5:
12            # Ascending sequence (with noise)
13            start = np.random.randint(0, vocab_size - seq_len - 5)
14            seq = np.sort(np.random.choice(range(start, start + seq_len + 5),
15                                         size=seq_len, replace=False))
16            x.append(seq)
17            y.append(1)
18        else:
19            # Random sequence
20            seq = np.random.randint(0, vocab_size, size=seq_len)
21            x.append(seq)
22            y.append(0)
23
24    x = torch.LongTensor(np.array(x))
25    y = torch.LongTensor(y)
26    return x, y
27
28
29 x_data, y_data = generate_sequence_data(3000)
30 n_train = 2400
31 x_train, y_train = x_data[:n_train], y_data[:n_train]
32 x_val, y_val = x_data[n_train:], y_data[n_train:]
33
34 print(f"Training: {x_train.shape}, Validation: {x_val.shape}")
35 print(f"Sample ascending: {x_data[y_data == 1][0].numpy()}")
36 print(f"Sample random: {x_data[y_data == 0][0].numpy()}")

```

In [3]

```

1 # =====
2 # Transformer Classifier
3 # =====
4
5 class PositionalEncoding(nn.Module):
6     """Sinusoidal positional encoding."""
7     def __init__(self, d_model, max_len=512):
8         super().__init__()
9         pe = torch.zeros(max_len, d_model)
10        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
11        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
12        pe[:, 0::2] = torch.sin(position * div_term)
13        pe[:, 1::2] = torch.cos(position * div_term)
14        self.register_buffer('pe', pe.unsqueeze(0)) # (1, max_len, d_model)
15
16    def forward(self, x):
17        return x + self.pe[:, :x.size(1), :]
18
19
20 class TransformerClassifier(nn.Module):
21     """Transformer encoder for sequence classification."""
22
23    def __init__(self, vocab_size, d_model=64, n_heads=4, n_layers=2,
24                 d_ff=128, n_classes=2, max_len=100, dropout=0.1):
25        super().__init__()
26
27        self.embedding = nn.Embedding(vocab_size, d_model)
28        self.pos_encoder = PositionalEncoding(d_model, max_len)
29        self.dropout = nn.Dropout(dropout)
30
31        # Transformer encoder layers
32        encoder_layer = nn.TransformerEncoderLayer(
33            d_model=d_model, nhead=n_heads, dim_feedforward=d_ff,
34            dropout=dropout, batch_first=True
35        )
36        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_layers)
37
38        # Classification head
39        self.classifier = nn.Sequential(
40            nn.Linear(d_model, d_model // 2),
41            nn.ReLU(),
42            nn.Dropout(dropout),
43            nn.Linear(d_model // 2, n_classes)
44        )
45
46    def forward(self, x):
47        # Embed tokens + add position
48        h = self.embedding(x) * math.sqrt(64)
49        h = self.pos_encoder(h)
50        h = self.dropout(h)
51
52        # Transformer encoder
53        h = self.transformer(h)
54
55        # Pool: mean of all token representations
56        h = h.mean(dim=1) # (batch, d_model)
57
58        # Classify

```

```

59         return self.classifier(h)
60
61
62 model = TransformerClassifier(vocab_size=50, d_model=64, n_heads=4,
63                             n_layers=2, d_ff=128, n_classes=2)
64 print(model)
65 print(f"\nTotal parameters: {sum(p.numel() for p in model.parameters()):,}")

```

In [4]

```

1  # =====
2  # Training loop
3  # =====
4  optimizer = optim.Adam(model.parameters(), lr=0.001)
5  criterion = nn.CrossEntropyLoss()
6
7  history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
8  batch_size = 64
9
10 for epoch in range(60):
11     model.train()
12     epoch_loss = 0
13     correct = 0
14     total = 0
15
16     # Mini-batch training
17     perm = torch.randperm(n_train)
18     for i in range(0, n_train, batch_size):
19         idx = perm[i:i+batch_size]
20         xb, yb = X_train[idx], y_train[idx]
21
22         logits = model(xb)
23         loss = criterion(logits, yb)
24
25         optimizer.zero_grad()
26         loss.backward()
27         optimizer.step()
28
29         epoch_loss += loss.item() * len(xb)
30         correct += (logits.argmax(1) == yb).sum().item()
31         total += len(xb)
32
33     # Validation
34     model.eval()
35     with torch.no_grad():
36         val_logits = model(X_val)
37         val_loss = criterion(val_logits, y_val).item()
38         val_acc = (val_logits.argmax(1) == y_val).float().mean().item()
39         train_acc = correct / total
40
41     history['train_loss'].append(epoch_loss / total)
42     history['val_loss'].append(val_loss)
43     history['train_acc'].append(train_acc)
44     history['val_acc'].append(val_acc)
45
46     if epoch % 10 == 0:
47         print(f" Epoch {epoch:3d} | Train Acc: {train_acc:.3f} | Val Acc: {val_acc:.3f}")
48
49 print(f"\nFinal Val Accuracy: {history['val_acc'][-1]:.1%}")

```

In [5]

```

1  # Visualize training
2  fig, axes = plt.subplots(1, 2, figsize=(12, 5))
3
4  axes[0].plot(history['train_loss'], label='Train', alpha=0.8)
5  axes[0].plot(history['val_loss'], label='Val', alpha=0.8)
6  axes[0].set_title('Loss')
7  axes[0].legend()
8  axes[0].grid(True, alpha=0.2)
9
10 axes[1].plot(history['train_acc'], label='Train', alpha=0.8)
11 axes[1].plot(history['val_acc'], label='Val', alpha=0.8)
12 axes[1].set_title('Accuracy')
13 axes[1].set_ylim(0.4, 1.05)
14 axes[1].legend()
15 axes[1].grid(True, alpha=0.2)
16
17 plt.suptitle('TRANSFORMER CLASSIFIER: Sequence Classification', fontsize=14, fontweight='bold')
18 plt.tight_layout()
19 plt.savefig('w5_02_training.png', dpi=100, bbox_inches='tight')
20 plt.show()

```

Part 2: Attention Visualization — What Does the Transformer See?

In [6]

```

1 # =====
2 # Extract and visualize attention weights
3 # =====
4 # We need to hook into the transformer to get attention
5
6 model.eval()
7 sample_idx = 0
8 x_sample = X_val[sample_idx:sample_idx+1]
9 y_true = y_val[sample_idx].item()
10
11 # Get prediction
12 with torch.no_grad():
13     logit = model(x_sample)
14     pred = logit.argmax(1).item()
15
16 print(f"Sequence: {x_sample[0].numpy()}")
17 print(f"True label: {'ascending' if y_true else 'random'}")
18 print(f"Prediction: {'ascending' if pred else 'random'}")
19
20 # For visualization, compute attention manually
21 with torch.no_grad():
22     h = model.embedding(x_sample) * math.sqrt(64)
23     h = model.pos_encoder(h)
24
25     # Get first layer's attention
26     layer0 = model.transformer.layers[0]
27     # Use the self_attn module directly
28     attn_out, attn_weights = layer0.self_attn(h, h, h, need_weights=True)
29
30 attn_np = attn_weights[0].numpy() # (seq_len, seq_len)
31 tokens = [str(t) for t in x_sample[0].numpy()]
32
33 fig, ax = plt.subplots(figsize=(7, 6))
34 im = ax.imshow(attn_np, cmap='Blues')
35 ax.set_xticks(range(len(tokens)))
36 ax.set_yticks(range(len(tokens)))
37 ax.set_xticklabels(tokens, fontsize=10)
38 ax.set_yticklabels(tokens, fontsize=10)
39 ax.set_xlabel('Keys')
40 ax.set_ylabel('Queries')
41 label = 'ascending' if y_true else 'random'
42 ax.set_title(f'Attention Weights (Layer 1) - "{label}" sequence')
43
44 for i in range(len(tokens)):
45     for j in range(len(tokens)):
46         ax.text(j, i, f'{attn_np[i,j]:.2f}', ha='center', va='center',
47                 fontsize=8, color='white' if attn_np[i,j] > 0.2 else 'black')
48
49 plt.colorbar(im, ax=ax)
50 plt.tight_layout()
51 plt.savefig('w5_02_attention_vis.png', dpi=100, bbox_inches='tight')
52 plt.show()

```

Part 3: Causal (Autoregressive) Masking — How GPT Works

GPT-style models can only attend to **past** tokens (not future ones). This is enforced with a **causal mask**: an upper-triangular matrix of -infinity.

In [7]

```

1 # =====
2 # Causal masking demonstration
3 # =====
4
5 seq_len = 6
6 tokens_demo = ["I", "love", "deep", "learn", "-ing", "!"]
7
8 # Causal mask: position i can only attend to positions <= i
9 causal_mask = torch.triu(torch.ones(seq_len, seq_len) * float('-inf'), diagonal=1)
10
11 print("Causal mask (GPT-style):")
12 print("0 = can attend, -inf = blocked")
13 for i in range(seq_len):
14     row = []
15     for j in range(seq_len):
16         row.append(" 0 " if causal_mask[i,j] == 0 else " -inf")
17     print(f" {tokens_demo[i]:6s}: {' '.join(row)}")
18
19 # Visualize
20 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
21
22 # Bidirectional (BERT-style)
23 bidir = np.ones((seq_len, seq_len))
24 axes[0].imshow(bidir, cmap='Blues', vmin=0, vmax=1)
25 axes[0].set_title('Bidirectional Attention (BERT)')
26 axes[0].set_xticks(range(seq_len))
27 axes[0].set_yticks(range(seq_len))
28 axes[0].set_xticklabels(tokens_demo, fontsize=9)
29 axes[0].set_yticklabels(tokens_demo, fontsize=9)
30
31 # Causal (GPT-style)
32 causal_vis = np.tril(np.ones((seq_len, seq_len)))
33 axes[1].imshow(causal_vis, cmap='Blues', vmin=0, vmax=1)
34 axes[1].set_title('Causal Attention (GPT)')
35 axes[1].set_xticks(range(seq_len))
36 axes[1].set_yticks(range(seq_len))
37 axes[1].set_xticklabels(tokens_demo, fontsize=9)
38 axes[1].set_yticklabels(tokens_demo, fontsize=9)
39
40 plt.suptitle('BERT vs GPT: Attention Masks', fontsize=14, fontweight='bold')
41 plt.tight_layout()
42 plt.savefig('w5_02_causal.png', dpi=100, bbox_inches='tight')
43 plt.show()
44
45 print("\nBERT: every token sees ALL other tokens (bidirectional)")
46 print("GPT: each token only sees PAST tokens (causal/autoregressive)")
47 print("This is the key architectural difference between BERT and GPT.")

```

Part 4: The Transformer Landscape — What's Out There

Model	Type	Attention	Key Use Case
BERT	Encoder	Bidirectional	Understanding (classification, NER)
GPT	Decoder	Causal	Generation (text, code)
T5	Encoder-Decoder	Both	Any text-to-text task
ViT	Encoder	Bidirectional	Image classification

CLIP	Dual Encoder	Cross-modal	Image-text matching
Whisper	Encoder-Decoder	Both	Speech recognition

The Transformer architecture is now the foundation of virtually all state-of-the-art models in NLP, vision, speech, and multimodal AI.

■ Week 5 Complete — Self-Assessment

You should now be able to:

- ☐ Build a Transformer encoder from scratch (pure Python AND PyTorch)
- ☐ Explain self-attention: "each token attends to all others via Q, K, V"
- ☐ Explain multi-head attention: "parallel attention for different relationships"
- ☐ Explain positional encoding: "sinusoidal signal that breaks permutation invariance"
- ☐ Distinguish BERT (bidirectional) from GPT (causal)
- ☐ Train a Transformer classifier in PyTorch
- ☐ Visualize attention weights to understand what the model learns

■ Congratulations — You Completed All 5 Weeks!

Your Learning Journey:

- **Week 1:** Neural networks from scratch — forward pass, backprop, ReLU
- **Week 2:** Deep networks in PyTorch — init, normalization, loss landscapes
- **Week 3:** Generative AI — VAEs, latent spaces, deployment
- **Week 4:** Graph Neural Networks — message passing, GCN, GraphSAGE
- **Week 5:** Transformers — attention, multi-head, positional encoding, GPT vs BERT

What's Next?

- **Fine-tune a pre-trained model** — Hugging Face makes this 10 lines of code
- **Diffusion Models** — the architecture behind Stable Diffusion, DALL-E
- **Reinforcement Learning from Human Feedback (RLHF)** — how ChatGPT is trained
- **Build something real** — take a problem you care about and apply what you've learned

"You've built every major architecture from scratch. Nothing in deep learning is magic anymore."